Performance Analysis Naive Matrix Multiplication vs. Tiled Matrix Multiplication

Joel Maldonado

March 7, 2026

1 Introduction

This report compares two GPU matrix multiplication kernels: a basic matrix multiplication implementation that accesses data directly from global memory, and a tiled matrix multiplication implementation that uses shared memory to reuse data within each thread block.

2 Experimental Setup

All experiments were executed in the same cluster. The hardware and software environment details are as follows:

- **GPU Model:** 1x NVIDIA Tesla P100 (16GB VRAM)
- **Compute Capability:** 6.0
- **CUDA Version:** 11.8
- **Host CPU Allocation:** 1 Core
- **Host RAM Allocation:** 6GB (6GB/core)

3 Results & Data

3.1 Execution Time Data

The experiment ran both kernels on four test cases and the configurations tests included thread block sizes of 4x4, 8x8, 16x16, and 32x32. Each kernel was executed 10 times per configuration to obtain average execution times and standard deviations. These results are summarized in Table 1 below. Additionally, note that for the tiled matrix multiplication algorithm, we set the MAX_TILE_SIZE to 32 to accommodate the largest block size tested.

Execution Time Data (ms)					
		Thread block configuration			
	Test case	**4x4**	**8x8**	**16x16**	**32x32**
Basic Matrix Multiply	Case 1 (2048x2048)	120.1 ± 1.9	58.4 ± 1.3	45.3 ± 0.0	43.2 ± 0.1
	Case 2 (2051x2051)	118.7 ± 1.7	58.5 ± 0.3	52.8 ± 0.1	56.1 ± 0.2
	Case 3 (4096x2048)	481.8 ± 2.3	235.9 ± 2.6	183.0 ± 3.2	171.3 ± 2.9
	Case 4 (4096x2047)	485.7 ± 2.7	242.9 ± 3.3	201.3 ± 3.5	214.5 ± 3.1
Tiled Matrix Multiply	Case 1 (2048x2048)	251.0 ± 0.0	38.4 ± 0.0	14.2 ± 0.0	11.1 ± 0.6
	Case 2 (2051x2051)	257.9 ± 3.4	38.2 ± 0.2	16.2 ± 0.1	12.1 ± 0.0
	Case 3 (4096x2048)	954.3 ± 8.9	143.5 ± 2.9	59.3 ± 3.4	45.8 ± 0.0
	Case 4 (4096x2047)	956.1 ± 6.8	144.6 ± 3.5	59.2 ± 2.8	45.8 ± 0.0

Table 1: Average kernel execution times over 10 iterations.

3.2 Speedup Analysis

To better quantify the benefit of shared-memory tiling, we compute the speedup of the tiled matrix multiplication algorithm relative to the basic matrix multiplication algorithm:

$$\text{Speedup} = \frac{T_{\text{basic}}}{T_{\text{tiled}}}$$

where T_{basic} and T_{tiled} are the execution times of the basic matrix multiplication algorithm and the tiled matrix multiplication algorithm respectively. Table 2 shows the speedup using the best-performing block configuration for both kernels.

Test Case	Speedup (Basic Algorithm / Tiled Algorithm)
Case 1 (2048×2048)	3.89×
Case 2 (2051×2051)	4.64×
Case 3 (4096×2048)	3.74×
Case 4 (4096×2047)	4.68×

Table 2: Speedup of the tiled matrix multiplication algorithm over the basic matrix multiplication algorithm using the 32×32 block configuration.

The tiled matrix multiplication algorithm achieves between approximately 3.7× and 4.7× speedup across the tested workloads. This improvement comes from reducing redundant global memory accesses. Each element loaded into shared memory is reused by multiple threads, which significantly lowers global memory bandwidth pressure.

3.3 Performance Visualizations

In this section we analyze the performance of both kernels across the different test cases and block sizes. Figures 1 analyzes the performance of the basic matrix multiplication algorithm. In Figure 2 we observe speedup of the tiled matrix multiplication algorithm, and in Figure 3 we compare the performance of both algorithms side by side on the largest test case, Case 4.

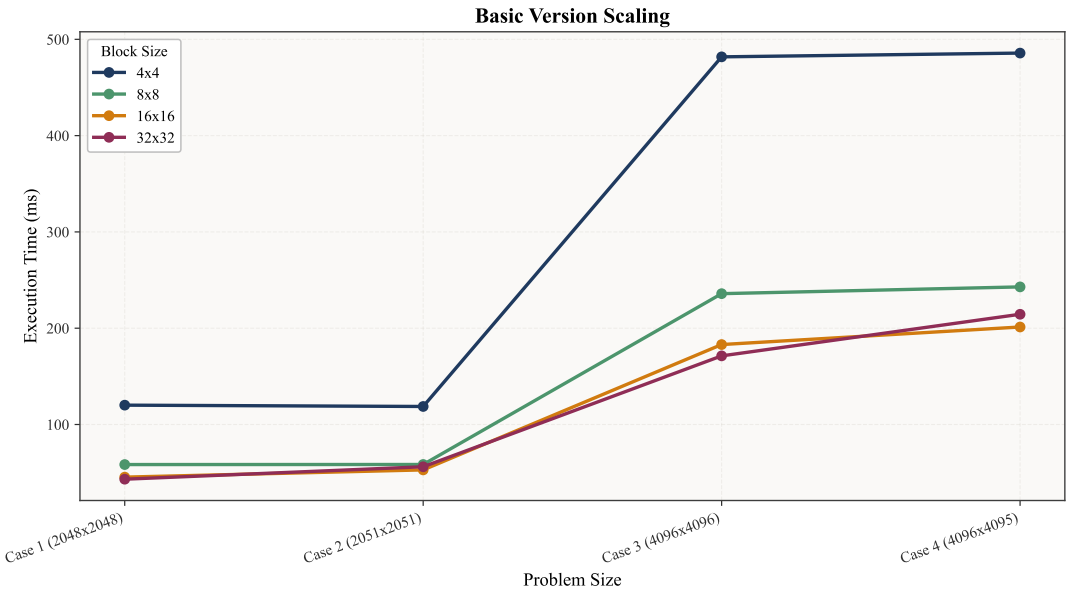

Figure 1: Execution time scaling for Basic Matrix Multiplication across block sizes.

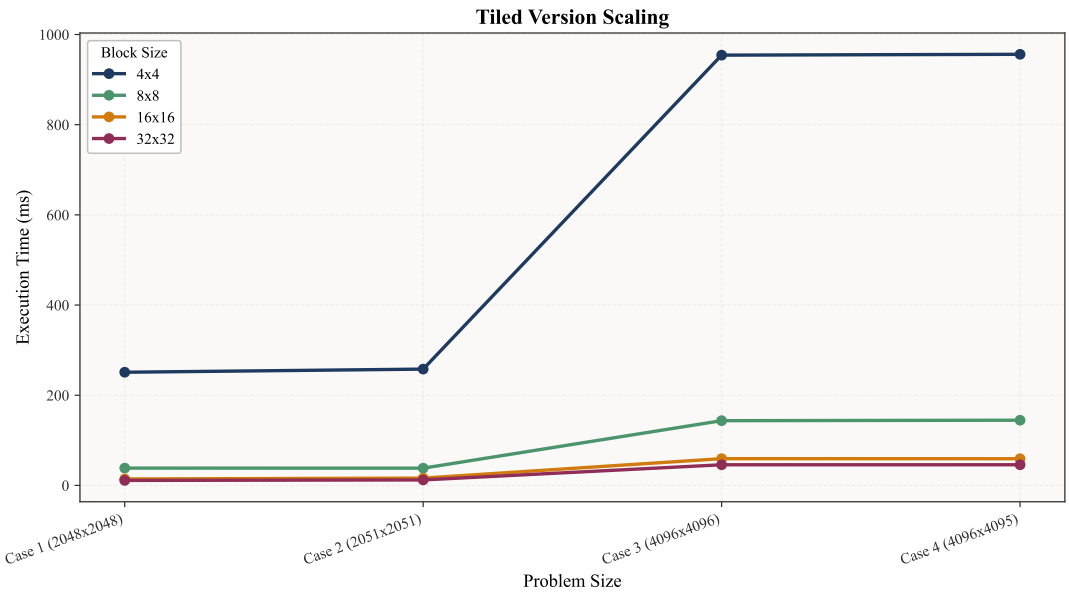

Figure 2: Execution time scaling for Tiled Matrix Multiplication across block sizes.

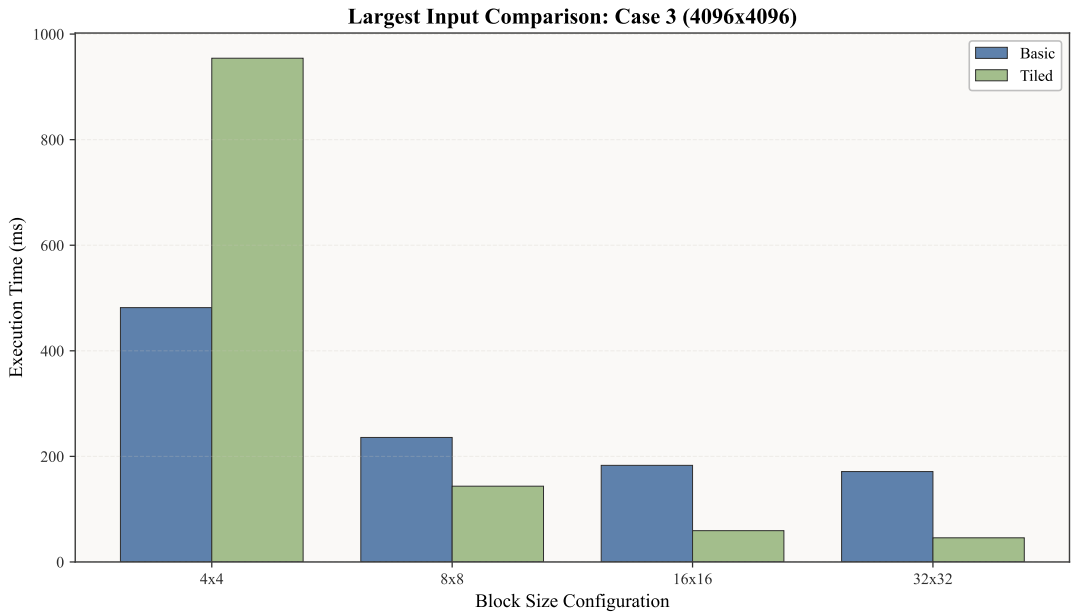

Figure 3: Basic matrix multiplication algorithm vs. tiled matrix multiplication algorithm on the largest plotted case.

4 Analysis & Discussion

This section answers the six required questions directly. Each answer includes both the observed trend and a hardware-level root-cause explanation.

Question 1

Question: Fix input size, discuss execution time trends with respect to change in thread block configuration for basic and tiled solutions individually. What is the optimal thread block configuration for each version? Discuss why a specific block size results with best/worst performance for each version. Does optimal block size depend on input size?
Answer: Fixix input size in Case 1, execution time falls as block size increases for both algorithms. The basic matrix multiplication algorithm drops from 120.1 ms at 4x4 to 43.2 ms at 32x32. The tiled matrix multiplication algorithm drops from 251.0 ms at 4x4 to 11.1 ms at 32x32. The worst configuration is 4x4 and the best is 32x32 for this case. The root cause is poor warp utilization at 4x4 and high synchronization overhead in the tiled matrix multiplication algorithm. The basic matrix multiplication algorithm is input dependent, since 16x16 outperforms 32x32 in Case 2 and Case 4. The tiled matrix multiplication algorithm remains best at 32x32 in the measured set.

Question 2

Question: Discuss execution time trends among square matrices only for basic and tiled versions individually.
Answer: For square matrices, Case 1 and Case 2 have nearly the same arithmetic workload. Even so, runtime increases are much larger than the arithmetic increase. At 32x32, the basic matrix multiplication algorithm rises from 43.2 ms to 56.1 ms, a 29.9% increase. The tiled matrix multiplication algorithm rises from 11.1 ms to 12.1 ms, a 9.0% increase. This behavior indicates memory and boundary effects, not pure compute scaling. The root cause is divergence and boundary inefficiency when dimensions are not clean multiples of the block size.

Question 3

Question: Discuss execution time trends among square and non-square matrices for basic and tiled versions individually.
Answer: Both algorithms slow down as problem size grows from the smaller cases to the larger cases. The key difference appears in shape sensitivity. At 32x32, the basic matrix multiplication algorithm increases from 171.3 ms in Case 3 to 214.5 ms in Case 4, which is a 25.2% increase. The tiled matrix multiplication algorithm stays essentially flat at about 45.8 ms. The root cause is that irregular dimensions create boundary blocks with partially inactive threads. The tiled matrix multiplication algorithm absorbs this cost better through shared-memory reuse.

Question 4

Question: Discuss performance of basic and tiled solution versions for a given problem size and across different problem sizes. Include any additional plot that will help your claims.
Answer: For a fixed problem size in Case 3, the basic matrix multiplication algorithm is faster only at 4x4. The tiled matrix multiplication algorithm is faster at 8x8, 16x16, and 32x32, with speedups of 1.64x, 3.09x, and 3.74x. Across all tested sizes, this ranking is consistent. Appendix Figure 4 and Appendix Figure 5 confirm the same trend over a wider workload range. The root cause is memory reuse. Shared-memory tiling reduces global memory transactions per output element and relieves bandwidth pressure.

Question 5

Question: Can basic version execute faster than the tiled version for the same problem size? You need to use dataset_generator to change the problem size and show execution time performance.
Answer: Yes. In the measured results, the basic matrix multiplication algorithm is faster than the tiled matrix multiplication algorithm at 4x4 for all tested problem sizes. In Case 1, the values are 120.1 ms and 251.0 ms. This occurs because tiling overhead is not amortized at very small work per block. For the requested extra sweep using `dataset_generator`, local execution was not possible because the available binaries are Linux ELF and the current shell is Windows.

Question 6

Question: Can basic version run faster than the tiled version for the same block size? Your discussion needs to be based on your experimental results.
Answer: Yes. At block size 4x4, the basic matrix multiplication algorithm is faster than the tiled matrix multiplication algorithm in all four cases. At 8x8, 16x16, and 32x32, the tiled matrix multiplication algorithm is consistently faster. The root cause is a synchronization-heavy execution path in the tiled matrix multiplication algorithm at 4x4. At larger blocks, shared-memory reuse dominates and the tiled matrix multiplication algorithm wins. The crossover occurs between 4x4 and 8x8.

5 Conclusion

The measured data supports three direct conclusions. First, 4x4 is consistently a poor configuration, especially for the tiled matrix multiplication algorithm. Second, the tiled matrix multiplication algorithm provides strong speedups at practical block sizes, reaching 3.74x in this dataset. Third, the tiled matrix multiplication algorithm is more robust to irregular dimensions than the basic matrix multiplication algorithm at larger scales. These results highlight a central principle of GPU programming: maximizing data reuse and memory locality often provides

larger performance gains than simply increasing raw computational throughput. In practice, this study shows that carefully chosen block sizes and shared-memory tiling are essential for achieving high performance on modern GPU architectures such as the NVIDIA Tesla P100.

A Appendix: Combined Scaling Plots

These appendix figures combine first-run cases and performance-analysis cases in a single log-log view to extend the visible workload range. They are intended to support the same conclusions, not replace the main analysis.

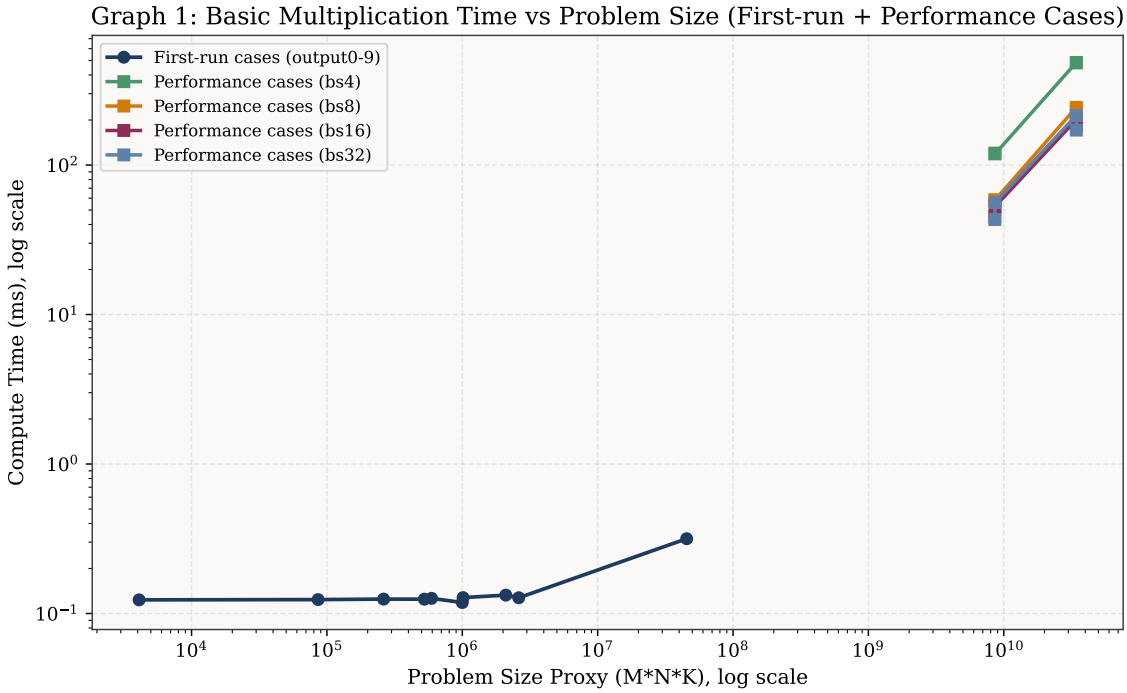

Figure 4: Appendix Graph A1: Basic multiplication time vs problem size (first-run + performance-analysis cases).

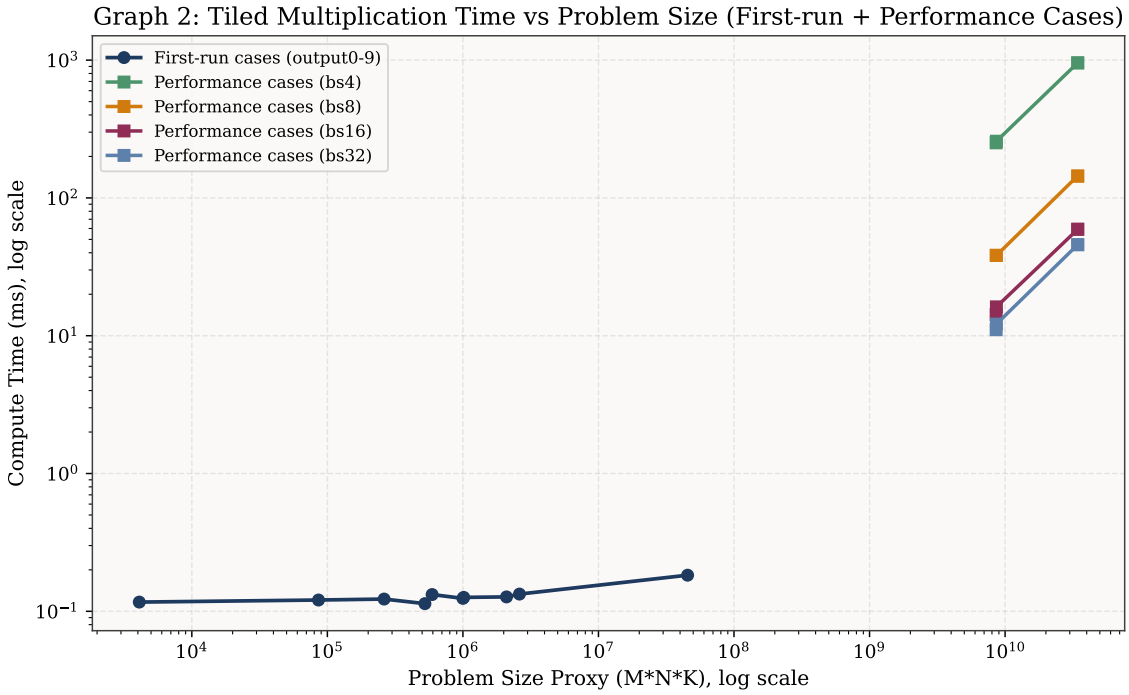

Figure 5: Appendix Graph A2: Tiled multiplication time vs problem size (first-run + performance-analysis cases).